

Electricity consumption UK 2009-2024 Analysis

Step 1: Title & Introduction

Electricity consumption UK 2009-2024 Analysis

Author: Muhammad Danish Azeem

Email: danishazeem@gmail.com

Dataset Source: [Electricity consumption UK 2009-2024](#)

Project Overview: Energy Demand & Generation Analysis

Introduction

Energy demand and generation play a crucial role in balancing power supply across a nation. Understanding the patterns of energy consumption, renewable energy contribution, and interconnector flows is essential for optimizing electricity distribution, preventing shortages, and ensuring a sustainable energy future.

Objective

The main goals of this project are:

- Analyze trends in national energy demand** over time.
- Evaluate the role of renewable energy sources** (wind and solar).
- Examine the impact of interconnectors** on energy imports and exports.
- Identify seasonal patterns and holiday effects** on electricity consumption.
- Develop predictive models** to forecast future energy demand and generation.

Dataset Overview

The dataset contains **time-series data** with key variables related to energy demand, renewable generation, and interconnector flows. Some important columns include:

- Settlement Date & Time:** When energy data was recorded.
- National Demand & Total System Demand:** Electricity consumption trends.

- **Embedded Wind & Solar Generation:** Renewable energy contributions.
- **Interconnector Flows:** Imports/exports of energy across borders.
- **Holiday & Seasonal Indicators:** Effects of holidays and seasons on demand.

▮ Methodology

▮ **Data Preprocessing:** Cleaning, handling missing values, and feature engineering.

▮ **Data Visualization:** Exploring demand trends, renewable generation patterns, and interconnector usage.

▮ **Predictive Analytics:** Using **Prophet (Time-Series Forecasting)** to predict future demand and energy generation.

▮ Why This Project Matters?

✂ Helps **electricity providers** optimize supply based on demand forecasts.

▮ Supports **renewable energy integration** for a greener future.

▮ Provides **valuable insights for energy policymakers & analysts**.

▮ Step 2: Importing Necessary Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from scipy.stats import zscore
import math
import os
import glob
from scipy.stats.mstats import winsorize
from scipy import stats
from google.colab import drive
```

▮ Step 3: Loading the Dataset

```
from google.colab import drive
drive.mount('/content/drive')
df = pd.read_csv('/content/drive/My Drive/Data Sets/Electricity
consumption UK 2009-2024.csv')
```

Mounted at /content/drive

Note: Some the output of notebook does not present the complete output, therefore we can increase the limit of columns view and row view by using these commands:

```
pd.set_option('display.max_columns', None) # this is to display all
the columns in the dataframe
pd.set_option('display.max_rows', None) # this is to display all the
rows in the dataframe

# hide all warnings runtime
import warnings
warnings.filterwarnings('ignore')
```

□ Step 4: Data Summary

```
# Display the first few rows
df.head()
```

```
{"type": "dataframe", "variable_name": "df"}
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 278512 entries, 0 to 278511
Data columns (total 19 columns):
```

#	Column	Non-Null	Count	Dtype
0	settlement_date	278512	non-null	object
1	settlement_period	278512	non-null	int64
2	period_hour	278512	non-null	object
3	nd	278512	non-null	int64
4	tsd	278512	non-null	int64
5	england_wales_demand	278512	non-null	int64
6	embedded_wind_generation	278512	non-null	int64
7	embedded_wind_capacity	278512	non-null	int64
8	embedded_solar_generation	278512	non-null	int64
9	embedded_solar_capacity	278512	non-null	int64
10	non_bm_stor	278512	non-null	int64
11	pump_storage_pumping	278512	non-null	int64
12	ifa_flow	278512	non-null	int64
13	ifa2_flow	278512	non-null	int64
14	britned_flow	278512	non-null	int64
15	moyle_flow	278512	non-null	int64
16	east_west_flow	278512	non-null	int64
17	nemo_flow	278512	non-null	int64
18	is_holiday	278512	non-null	int64

```
dtypes: int64(17), object(2)
memory usage: 40.4+ MB
```

Observations

1. There are 278512 rows and 19 columns in the dataset
 2. We see that settlement_date and period_hour are stored as objects (strings). We need to convert them into appropriate formats.
 3. There are No missing values in this dataset.
-

```
df.sample(50)
```

```
{"repr_error": "0", "type": "dataframe"}
```

- `df.sample(50)` gives us whole picture or idea about data.

```
df.columns
```

```
Index(['settlement_date', 'settlement_period', 'period_hour', 'nd',  
      'tsd',  
      'england_wales_demand', 'embedded_wind_generation',  
      'embedded_wind_capacity', 'embedded_solar_generation',  
      'embedded_solar_capacity', 'non_bm_stor',  
      'pump_storage_pumping',  
      'ifa_flow', 'ifa2_flow', 'britned_flow', 'moyle_flow',  
      'east_west_flow',  
      'nemo_flow', 'is_holiday'],  
      dtype='object')
```

- let's see the exact column names which can be easily copied later

```
# Descriptive statistics
```

```
df.describe(include='all')
```

```
{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 11,\n  \"fields\": [\n    {\n      \"column\": \"settlement_date\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"1970-01-01 00:00:00.000000001\",\n        \"max\": \"2009-01-01 00:00:00\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"278512\",\n          \"2009-01-01 00:00:00\",\n          \"1\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"settlement_period\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 98460.81463940734,\n        \"min\": 1.0,\n        \"max\": 278512.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          24.497357384960072,\n          24.0,\n          278512.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"period_hour\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"1970-01-01 00:00:00.000000048\",\n        \"max\": \"2025-03-01 00:00:00\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          48,\n          \"5803\",\n          \"278512\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}
```

```
\\"description\\": \\"\\\\"\\n    }\\n    },\\n    {\\n        \\"column\\":  
\\nd\\",\\n        \\"properties\\": {\\n            \\"dtype\\": \\"number\\",\\n  
\\std\\": 89529.92741664298,\\n            \\"min\\": 7831.306542443342,\\n  
\\max\\": 278512.0,\\n            \\"num_unique_values\\": 8,\\n  
\\samples\\": [\\n                31187.04136985121,\\n                30491.0,\\n  
278512.0\\n            ],\\n            \\"semantic_type\\": \\"\\",\\n  
\\description\\": \\"\\\\"\\n        }\\n    },\\n    {\\n        \\"column\\":  
\\tsd\\",\\n        \\"properties\\": {\\n            \\"dtype\\": \\"number\\",\\n  
\\std\\": 89044.13666478236,\\n            \\"min\\": 7598.781278849585,\\n  
\\max\\": 278512.0,\\n            \\"num_unique_values\\": 8,\\n  
\\samples\\": [\\n                32685.811142787385,\\n                31800.0,\\n  
278512.0\\n            ],\\n            \\"semantic_type\\": \\"\\",\\n  
\\description\\": \\"\\\\"\\n        }\\n    },\\n    {\\n        \\"column\\":  
\\england_wales_demand\\",\\n        \\"properties\\": {\\n            \\"dtype\\":  
\\number\\",\\n            \\"std\\": 91196.11608595274,\\n            \\"min\\":  
0.0,\\n            \\"max\\": 278512.0,\\n            \\"num_unique_values\\": 8,\\n  
\\samples\\": [\\n                28389.589216981673,\\n                27753.0,\\n  
278512.0\\n            ],\\n            \\"semantic_type\\": \\"\\",\\n  
\\description\\": \\"\\\\"\\n        }\\n    },\\n    {\\n        \\"column\\":  
\\embedded_wind_generation\\",\\n        \\"properties\\": {\\n  
\\dtype\\": \\"number\\",\\n            \\"std\\": 97906.63077657297,\\n  
\\min\\": 0.0,\\n            \\"max\\": 278512.0,\\n  
\\num_unique_values\\": 8,\\n            \\"samples\\": [\\n  
1272.0850268570116,\\n                1018.0,\\n                278512.0\\n  
n            ],\\n            \\"semantic_type\\": \\"\\",\\n  
\\description\\": \\"\\\\"\\n        }\\n    },\\n    {\\n        \\"column\\":  
\\embedded_wind_capacity\\",\\n        \\"properties\\": {\\n  
\\dtype\\": \\"number\\",\\n            \\"std\\": 97080.69391859311,\\n  
\\min\\": 1403.0,\\n            \\"max\\": 278512.0,\\n  
\\num_unique_values\\": 8,\\n            \\"samples\\": [\\n  
4453.970263399781,\\n                4831.0,\\n                278512.0\\n  
n            ],\\n            \\"semantic_type\\": \\"\\",\\n            \\"description\\": \\"\\\\"\\n  
}\\n    },\\n    {\\n        \\"column\\": \\"embedded_solar_generation\\",\\n  
\\properties\\": {\\n            \\"dtype\\": \\"number\\",\\n            \\"std\\":  
97798.94636014198,\\n            \\"min\\": 0.0,\\n            \\"max\\": 278512.0,\\n  
n            \\"num_unique_values\\": 6,\\n            \\"samples\\": [\\n  
278512.0,\\n                880.2453143850175,\\n                11224.0\\n  
n            ],\\n            \\"semantic_type\\": \\"\\",\\n  
\\description\\": \\"\\\\"\\n        }\\n    },\\n    {\\n        \\"column\\":  
\\embedded_solar_capacity\\",\\n        \\"properties\\": {\\n  
\\dtype\\": \\"number\\",\\n            \\"std\\": 95688.28090586832,\\n  
\\min\\": 0.0,\\n            \\"max\\": 278512.0,\\n  
\\num_unique_values\\": 8,\\n            \\"samples\\": [\\n  
8705.061548514965,\\n                11503.0,\\n                278512.0\\n  
n            ],\\n            \\"semantic_type\\": \\"\\",\\n  
\\description\\": \\"\\\\"\\n        }\\n    },\\n    {\\n        \\"column\\":  
\\non_bm_stor\\",\\n        \\"properties\\": {\\n            \\"dtype\\":  
\\number\\",\\n            \\"std\\": 98423.14773103037,\\n            \\"min\\": -  
24.0,\\n            \\"max\\": 278512.0,\\n            \\"num unique values\\": 6,\\n
```

```

\"samples\": [\n          278512.0,\n          6.6684164416614005,\n          893.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"column\":\n        \"pump_storage_pumping\",\n        \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 98307.62785472664,\n          \"min\":\n          0.0,\n          \"max\": 278512.0,\n          \"num_unique_values\": 8,\n          \"samples\": [\n            304.49703064858966,\n            11.0,\n            278512.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\":\n          \"ifa_flow\",\n          \"properties\": {\n            \"dtype\":\n            \"number\",\n            \"std\": 98206.92480298555,\n            \"min\": -\n            2056.0,\n            \"max\": 278512.0,\n            \"num_unique_values\":\n            8,\n            \"samples\": [\n              926.2476087206296,\n              1183.0,\n              278512.0\n            ],\n            \"semantic_type\":\n            \"\",\n            \"description\": \"\",\n            \"column\":\n            \"ifa2_flow\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 98450.54859179485,\n              \"min\": -1030.0,\n              \"max\": 278512.0,\n              \"num_unique_values\": 6,\n              \"samples\": [\n                278512.0,\n                53.998768455219164,\n                1016.0\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\",\n              \"column\":\n              \"britned_flow\",\n              \"properties\": {\n                \"dtype\":\n                \"number\",\n                \"std\": 98335.57022599177,\n                \"min\": -\n                1215.0,\n                \"max\": 278512.0,\n                \"num_unique_values\":\n                8,\n                \"samples\": [\n                  504.1379976446257,\n                  726.0,\n                  278512.0\n                ],\n                \"semantic_type\":\n                \"\",\n                \"description\": \"\",\n                \"column\":\n                \"moyle_flow\",\n                \"properties\": {\n                  \"dtype\": \"number\",\n                  \"std\": 98485.85313334805,\n                  \"min\": -505.0,\n                  \"max\": 278512.0,\n                  \"num_unique_values\": 8,\n                  \"samples\": [\n                    125.14527560751422,\n                    -148.0,\n                    278512.0\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\",\n                  \"column\":\n                  \"east_west_flow\",\n                  \"properties\": {\n                    \"dtype\":\n                    \"number\",\n                    \"std\": 98473.30912432105,\n                    \"min\": -\n                    585.0,\n                    \"max\": 278512.0,\n                    \"num_unique_values\": 7,\n                    \"samples\": [\n                      278512.0,\n                      -\n                      51.10674225886138,\n                      0.0\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\",\n                    \"column\":\n                    \"nemo_flow\",\n                    \"properties\": {\n                      \"dtype\": \"number\",\n                      \"std\":\n                      98422.73393535707,\n                      \"min\": -1023.0,\n                      \"max\":\n                      278512.0,\n                      \"num_unique_values\": 7,\n                      \"samples\": [\n                        278512.0,\n                        176.33687596943759,\n                        338.0\n                      ],\n                      \"semantic_type\": \"\",\n                      \"description\": \"\",\n                      \"column\":\n                      \"is_holiday\",\n                      \"properties\": {\n                        \"dtype\": \"number\",\n                        \"std\":

```

```
98468.80280733563,\n          \"min\": 0.0,\n          \"max\": 278512.0,\n          \"num_unique_values\": 5,\n          \"samples\": [\n0.022404779686333084,\n          1.0,\n          0.14799623027677697\n],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n}\n    }\n    ]\n}","type":"dataframe"}
```

Step 5: Handling Data Types

```
# Convert 'settlement_date' to datetime format
df['settlement_date'] = pd.to_datetime(df['settlement_date'])

# Convert 'period_hour' to time format
df['period_hour'] = pd.to_datetime(df['period_hour'], format='%H:%M:%S').dt.time
```

Explanation:

- `pd.to_datetime(df['settlement_date'])` ensures that the date is correctly recognized.
- `pd.to_datetime(df['period_hour'], format='%H:%M:%S').dt.time` converts time properly.

Step 6: Feature Renaming

To improve readability, we rename columns.

```
df.rename(columns={
    'nd': 'national_demand',
    'tsd': 'total_system_demand',
    'ifa_flow': 'interconnector_ifa',
    'ifa2_flow': 'interconnector_ifa2',
    'britned_flow': 'interconnector_britned',
    'moyle_flow': 'interconnector_moyle',
    'east_west_flow': 'interconnector_east_west',
    'nemo_flow': 'interconnector_nemo'
}, inplace=True)
```

Explanation:

- This renames some columns for better clarity.

Step 7: Creating New Features

We can extract useful time-based features.

```
# Extract year, month, day, and hour
df['year'] = df['settlement_date'].dt.year
df['month'] = df['settlement_date'].dt.month
df['day'] = df['settlement_date'].dt.day
df['weekday'] = df['settlement_date'].dt.weekday
df['hour'] = pd.to_datetime(df['period_hour'], format='%H:%M:%S').dt.hour
```

Explanation:

- Extracting time-based features helps in analyzing trends.

#Step 8: Handling Duplicates Checking for duplicate rows.

```
# Check for duplicates
duplicates = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicates}")

Number of duplicate rows: 0
```

□ Exploratory Data Analysis (EDA)

```
df.describe(include='all')

{"type": "dataframe"}
```

Step 9: Saving the Cleaned Data

After preprocessing, we save the dataset.

```
df.to_csv('/content/drive/My Drive/Data
Sets/cleaned_electricity_consumption_uk.csv', index=False)
```

After preprocessing, we saved the cleaned DataFrame as a new CSV.

□ Step 10: Data Visualization & Key Insights

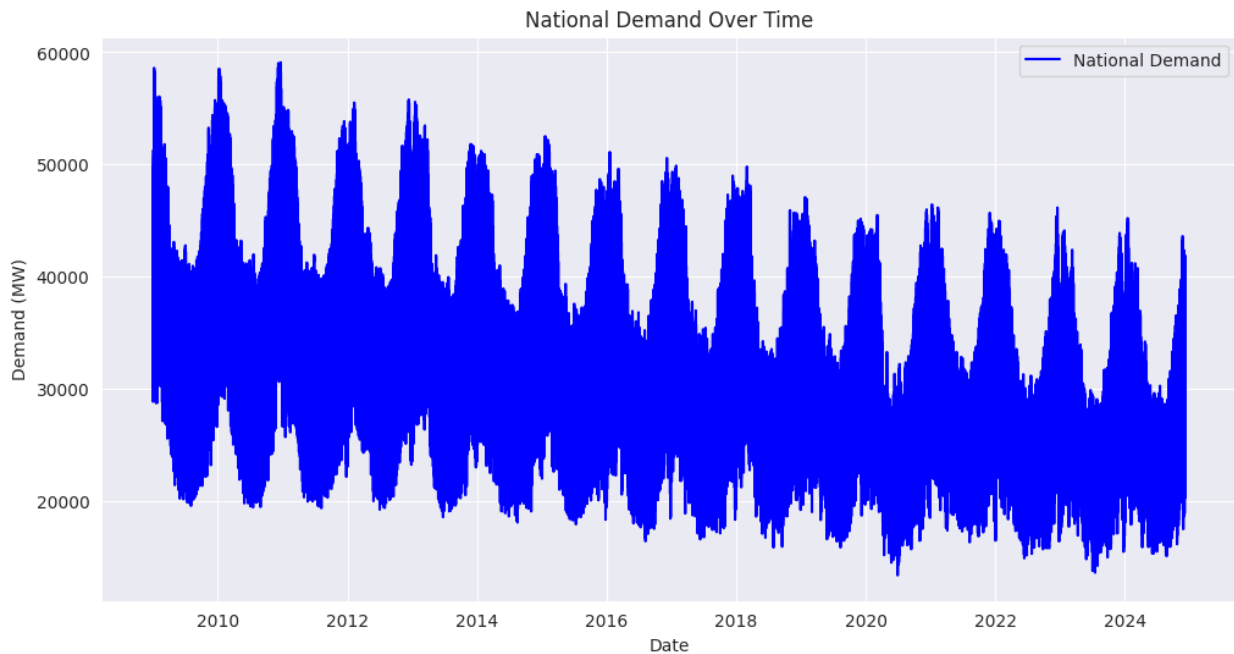
1. Time-Series Plot: National Demand Over Time

```
# Set plot style
sns.set_style("darkgrid")

### --- 1. Time-Series Plot: National Demand Over Time ---
plt.figure(figsize=(12, 6))
```

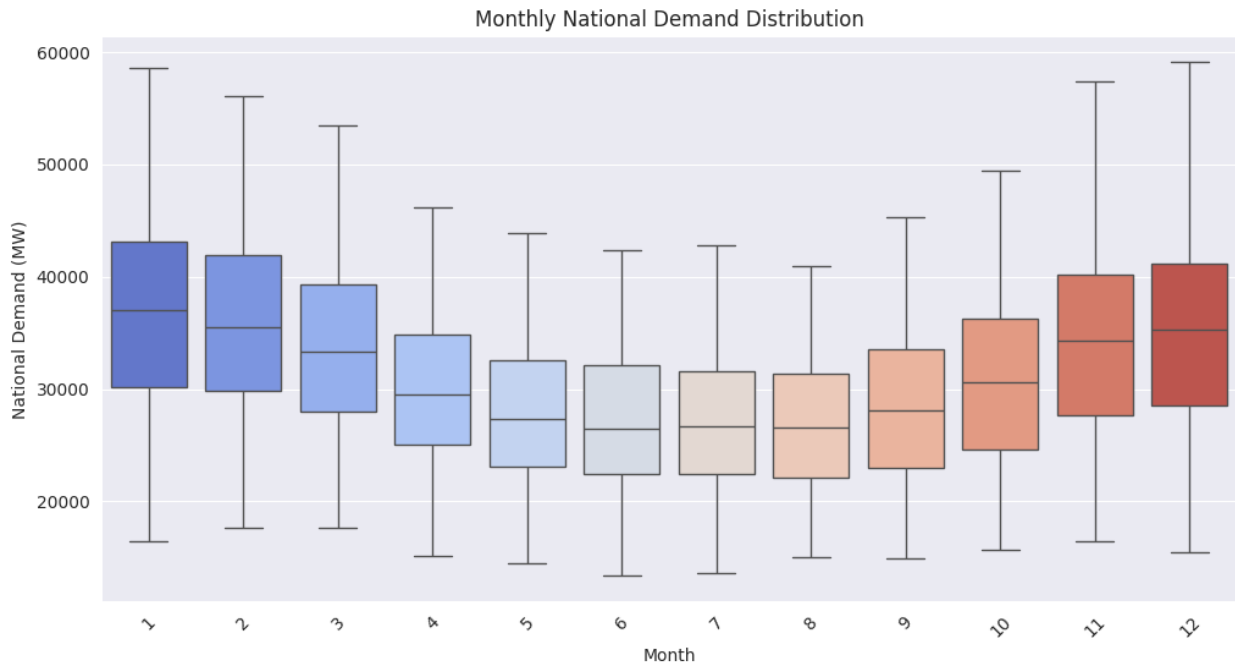


```
plt.plot(df['settlement_date'], df['national_demand'], label="National Demand", color='blue')
plt.xlabel("Date")
plt.ylabel("Demand (MW)")
plt.title("National Demand Over Time")
plt.legend()
plt.show()
```



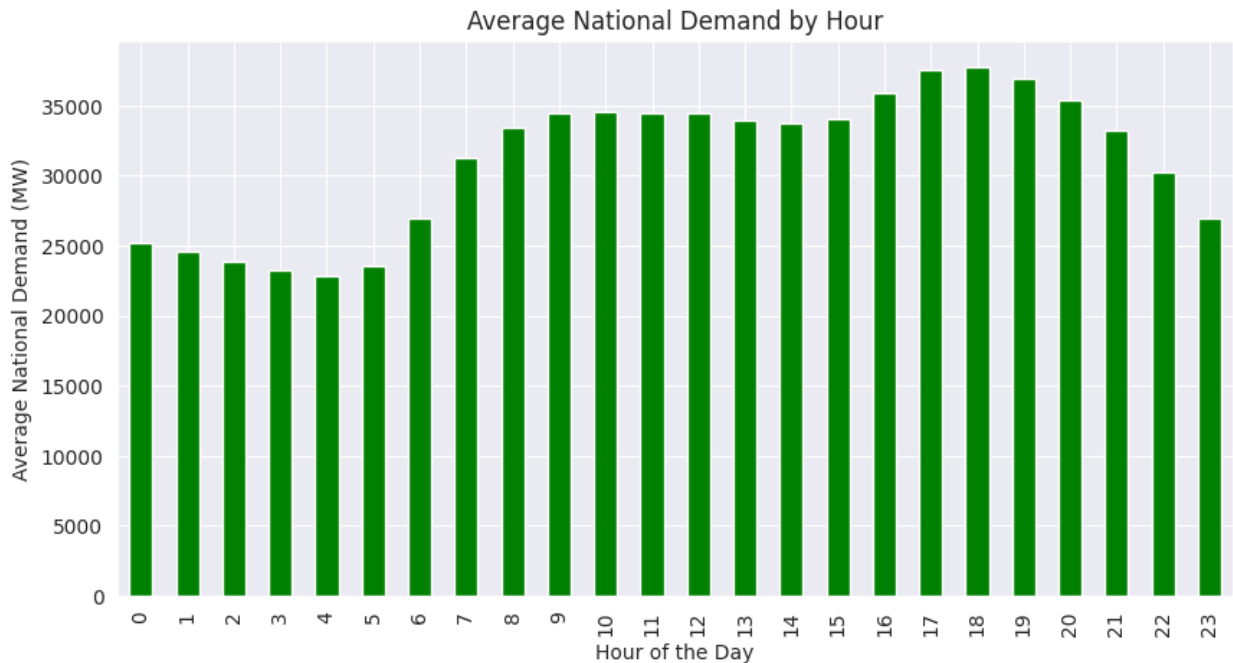
2. Monthly Demand Trends (Boxplot)

```
### --- 2. Monthly Demand Trends (Boxplot) ---
plt.figure(figsize=(12, 6))
sns.boxplot(x=df['month'], y=df['national_demand'],
palette="coolwarm")
plt.xlabel("Month")
plt.ylabel("National Demand (MW)")
plt.title("Monthly National Demand Distribution")
plt.xticks(rotation=45)
plt.show()
```



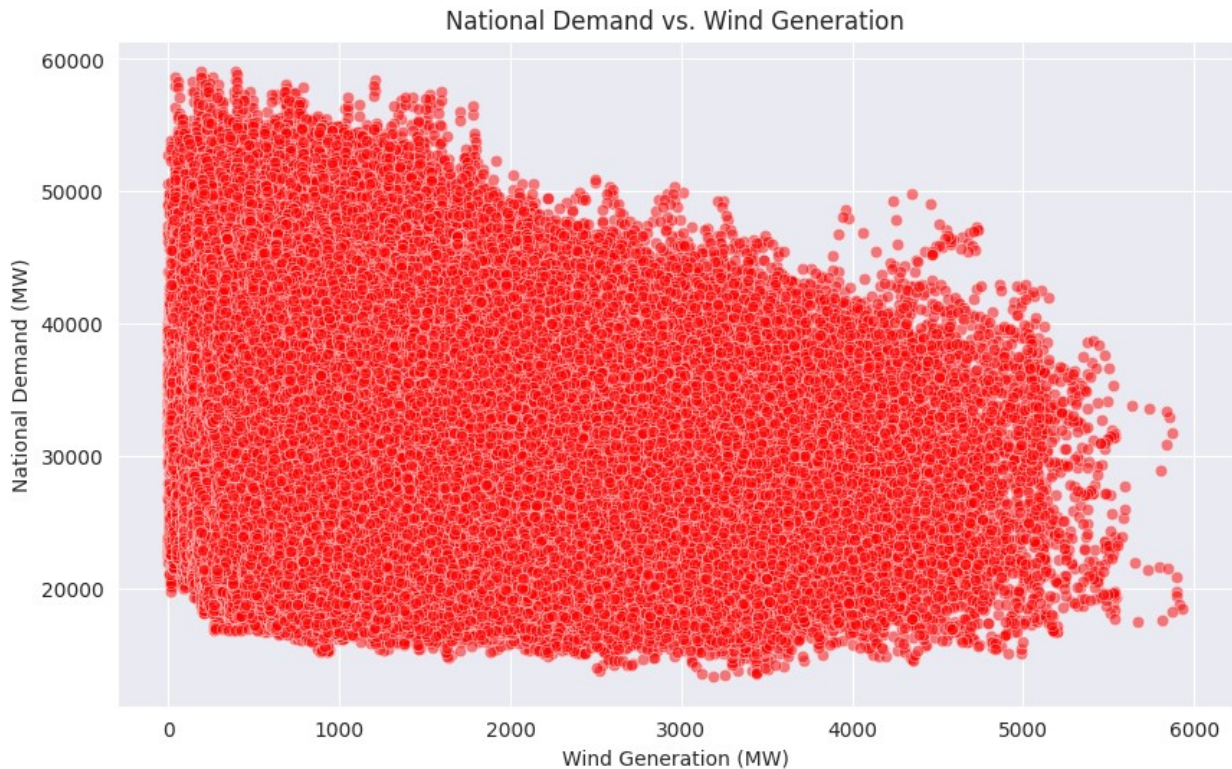
3. Daily Demand Patterns (Average Demand by Hour)

```
### --- 3. Daily Demand Patterns (Average Demand by Hour) ---
plt.figure(figsize=(10, 5))
df.groupby('hour')['national_demand'].mean().plot(kind='bar',
color='green')
plt.xlabel("Hour of the Day")
plt.ylabel("Average National Demand (MW)")
plt.title("Average National Demand by Hour")
plt.show()
```



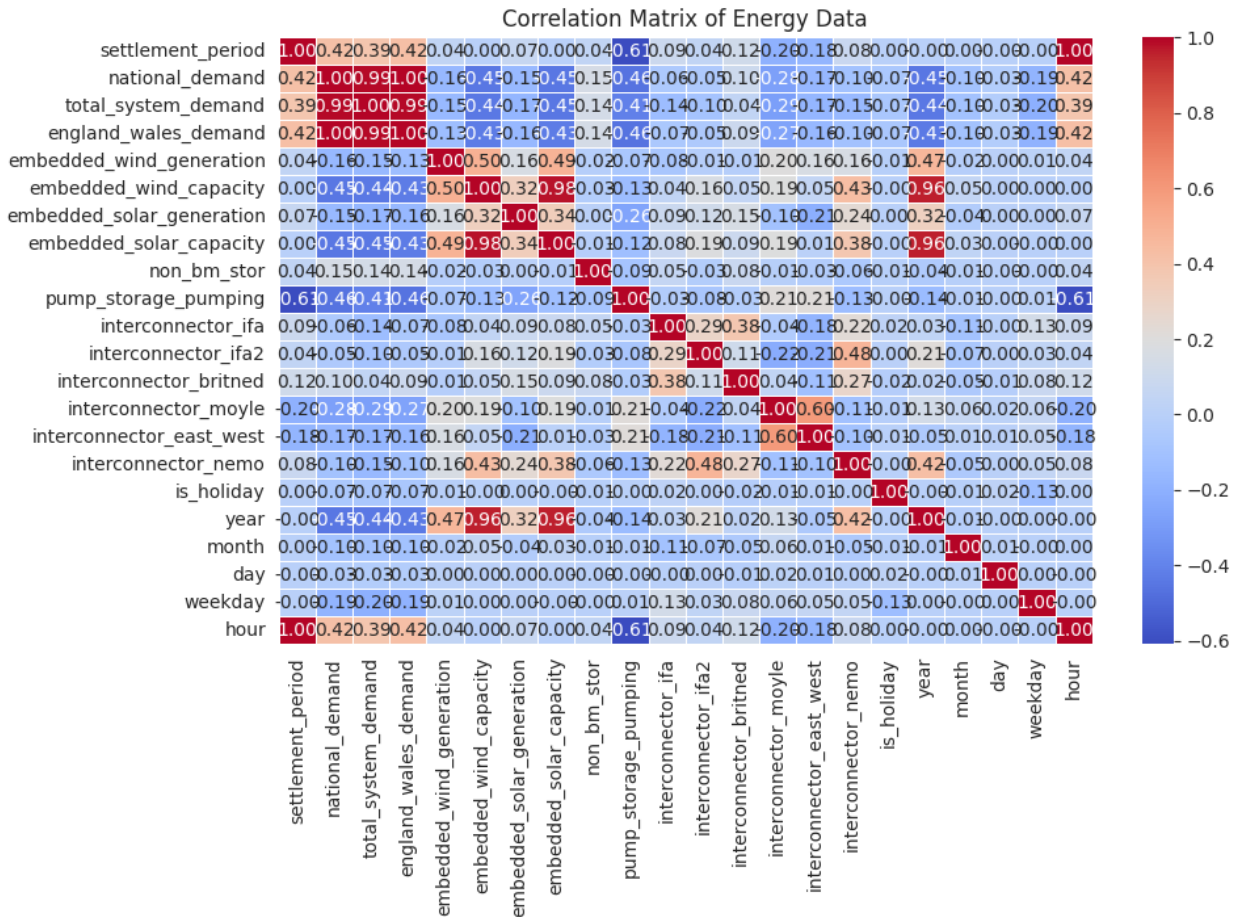
4. Demand vs. Renewable Energy Generation (Scatter Plot)

```
### --- 4. Demand vs. Renewable Energy Generation (Scatter Plot) ---
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df["embedded_wind_generation"],
y=df["national_demand"], color="red", alpha=0.5)
plt.xlabel("Wind Generation (MW)")
plt.ylabel("National Demand (MW)")
plt.title("National Demand vs. Wind Generation")
plt.show()
```



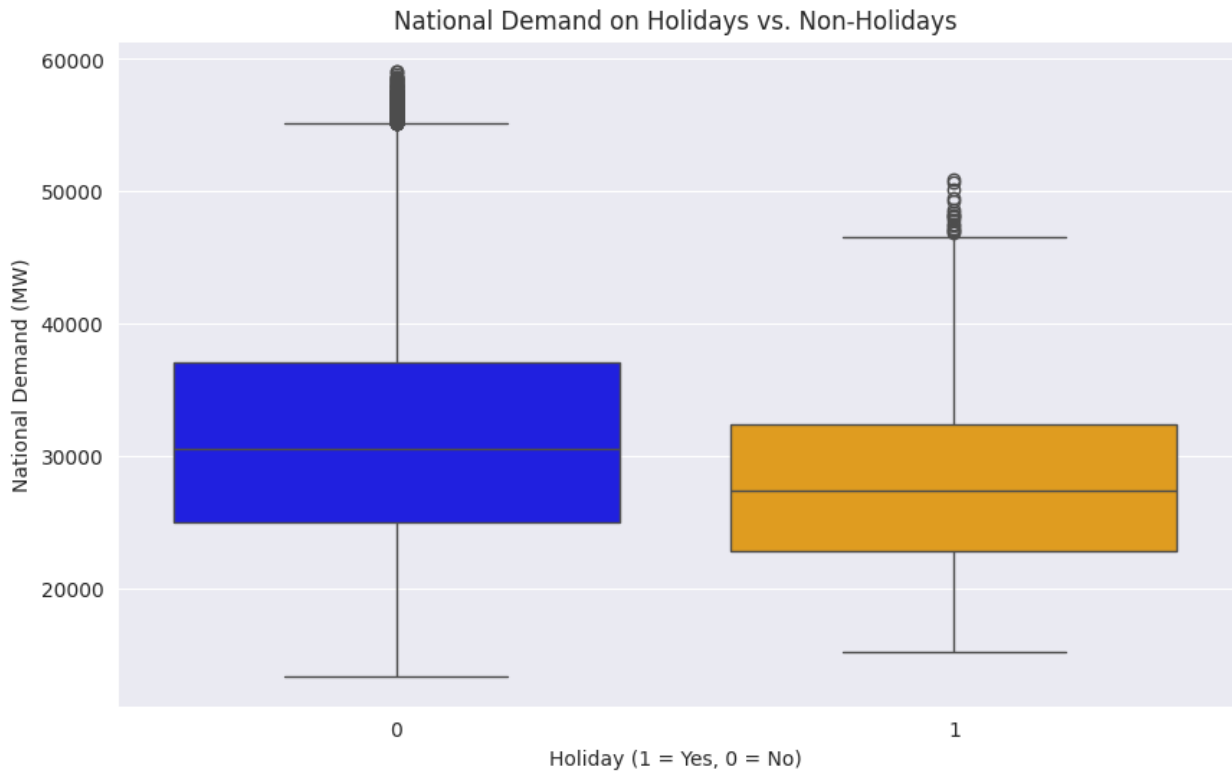
5. Correlation Heatmap

```
# --- 5. Correlation Heatmap ---  
# Drop non-numeric columns before correlation analysis  
df_numeric = df.select_dtypes(include=["number"]) # Keep only  
numerical columns  
  
plt.figure(figsize=(10, 6))  
sns.heatmap(df_numeric.corr(), annot=True, cmap="coolwarm", fmt=".2f",  
linewidths=0.5)  
plt.title("Correlation Matrix of Energy Data")  
plt.show()
```



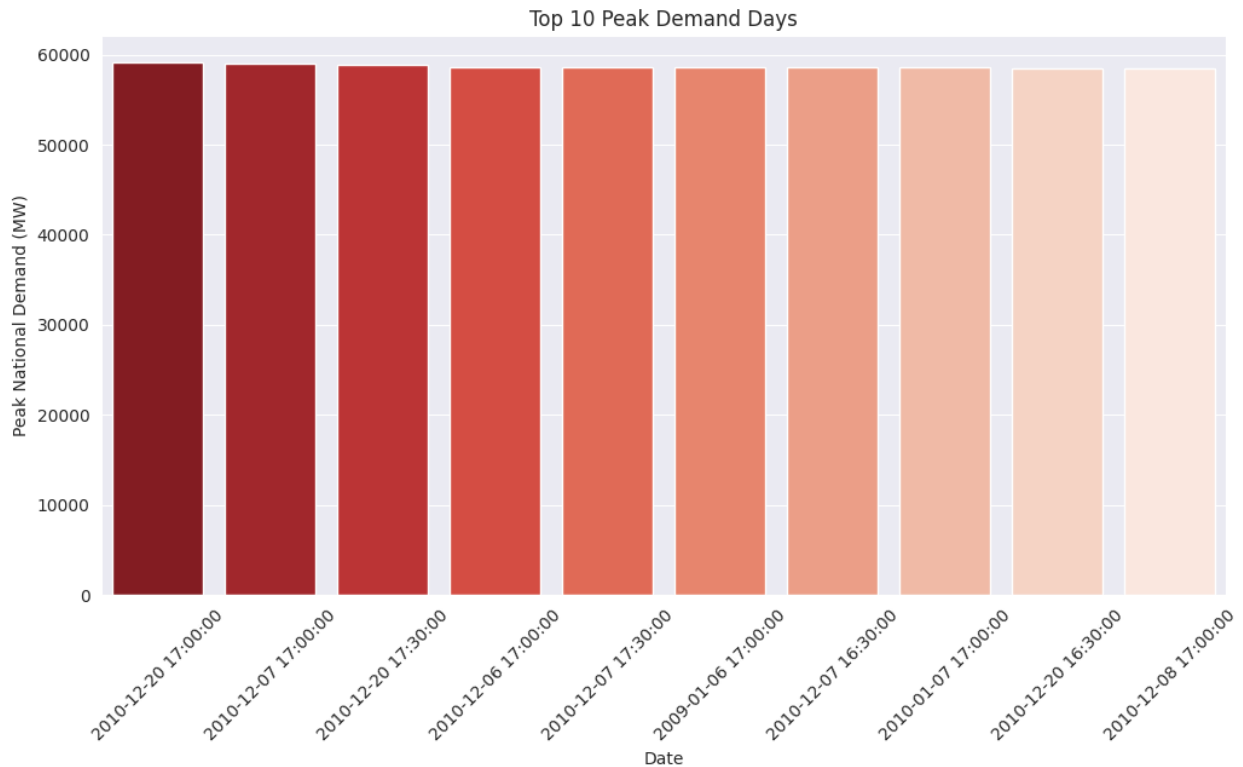
6. Holiday vs. Non-Holiday Demand (Boxplot)

```
### --- 6. Holiday vs. Non-Holiday Demand (Boxplot) ---
plt.figure(figsize=(10, 6))
sns.boxplot(x=df["is_holiday"], y=df["national_demand"],
palette=["blue", "orange"])
plt.xlabel("Holiday (1 = Yes, 0 = No)")
plt.ylabel("National Demand (MW)")
plt.title("National Demand on Holidays vs. Non-Holidays")
plt.show()
```



7. Peak Demand Days

```
### --- 7. Peak Demand Days ---
top_demand = df.nlargest(10, "national_demand")
plt.figure(figsize=(12, 6))
sns.barplot(x=top_demand["settlement_date"],
y=top_demand["national_demand"], palette="Reds_r")
plt.xlabel("Date")
plt.ylabel("Peak National Demand (MW)")
plt.title("Top 10 Peak Demand Days")
plt.xticks(rotation=45)
plt.show()
```

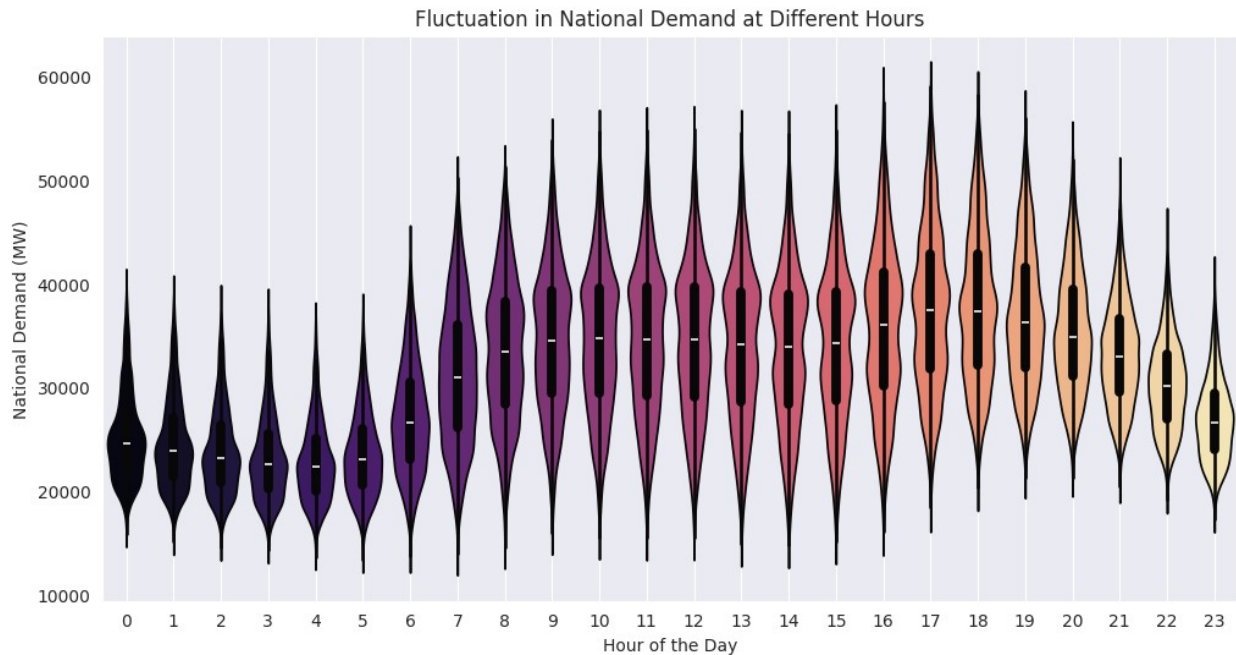


8. Energy Demand Fluctuations Over the Day (Violin Plot)

This shows the **distribution** of energy demand at different hours.

```
plt.figure(figsize=(12, 6))
sns.violinplot(x="hour", y="national_demand", data=df,
palette="magma")

plt.title("Fluctuation in National Demand at Different Hours")
plt.xlabel("Hour of the Day")
plt.ylabel("National Demand (MW)")
plt.grid()
plt.show()
```

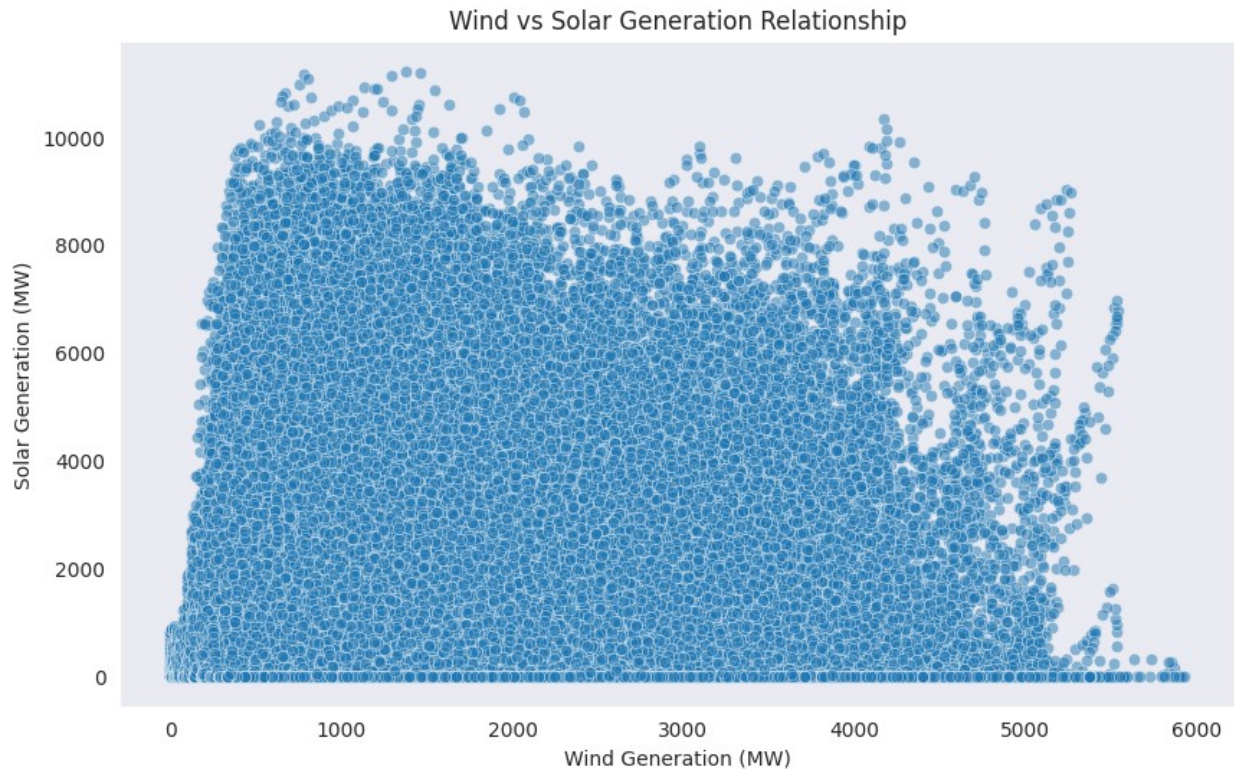


9. Solar vs Wind Generation Relationship (Scatter Plot)

Check if **wind** and **solar generation** are correlated.

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df["embedded_wind_generation"],
y=df["embedded_solar_generation"], alpha=0.5)

plt.title("Wind vs Solar Generation Relationship")
plt.xlabel("Wind Generation (MW)")
plt.ylabel("Solar Generation (MW)")
plt.grid()
plt.show()
```

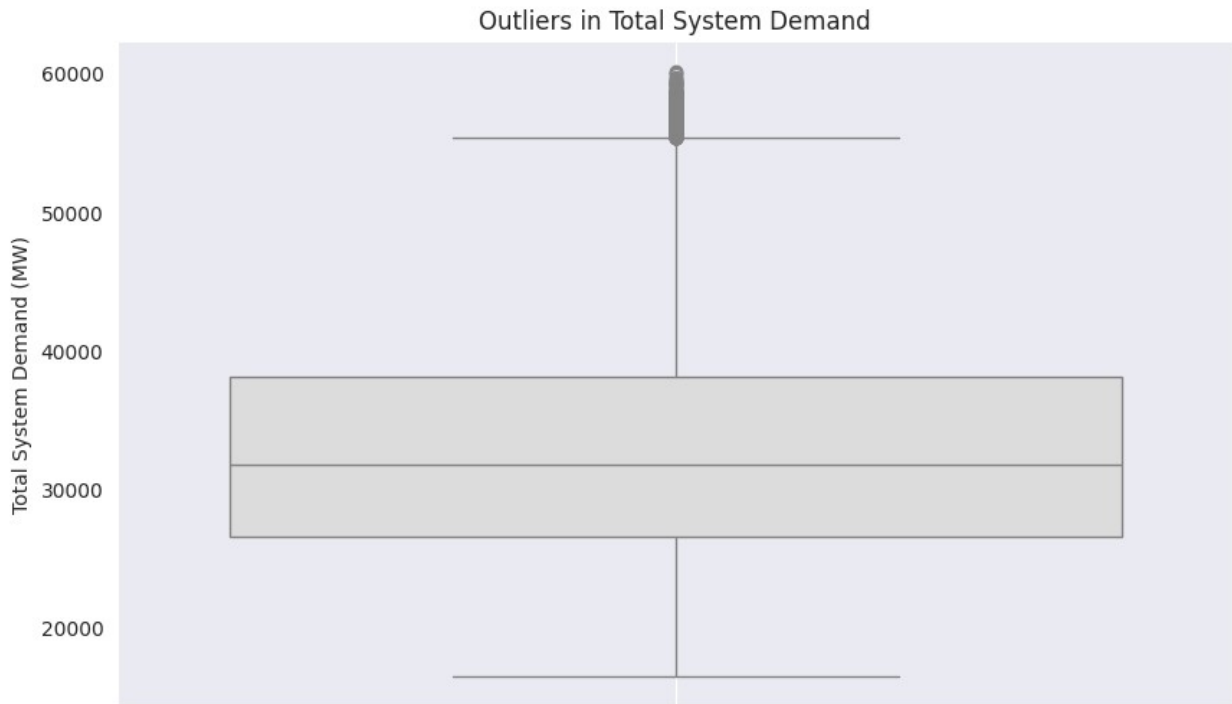



10. Energy Demand Anomalies (Boxplot of Outliers)

Find **unusual spikes or drops** in total system demand.

```
plt.figure(figsize=(10, 6))
sns.boxplot(y="total_system_demand", data=df, palette="coolwarm")

plt.title("Outliers in Total System Demand")
plt.ylabel("Total System Demand (MW)")
plt.grid()
plt.show()
```



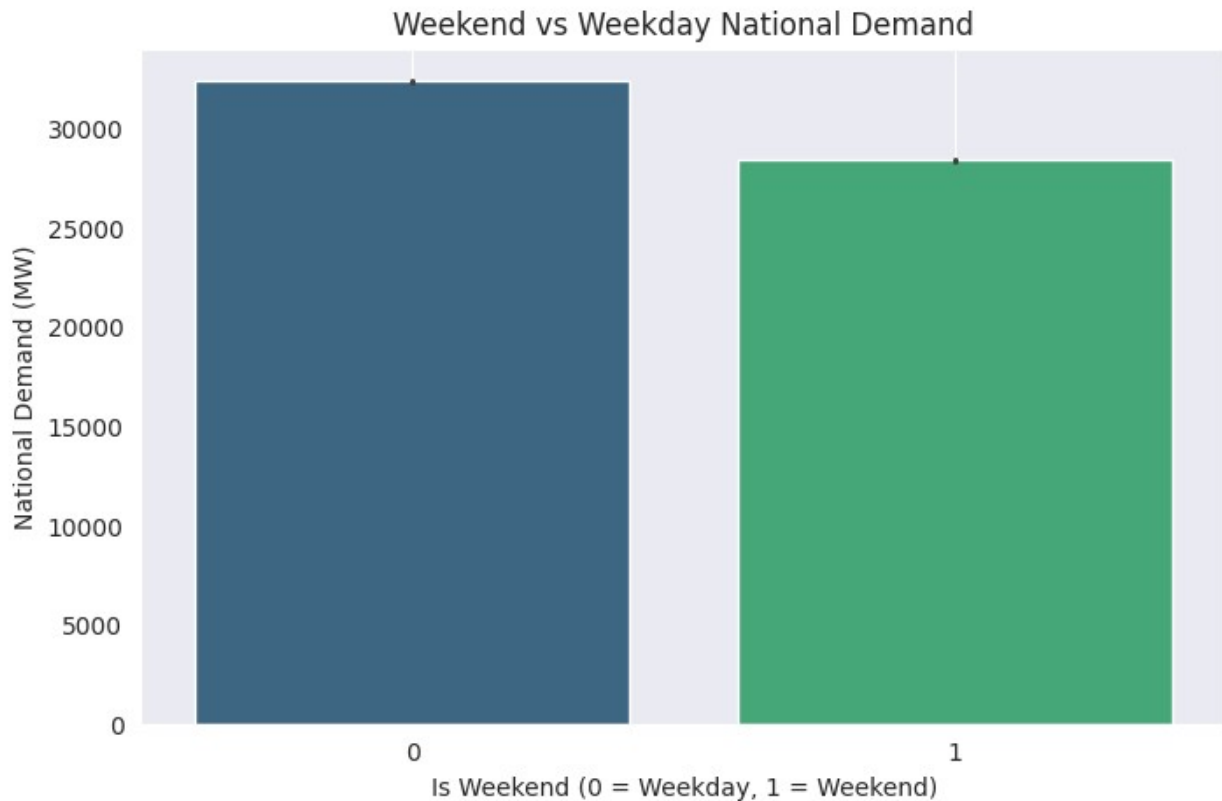
11. Weekend vs Weekday Demand (Bar Plot)

Compare average demand between **weekdays** and **weekend**

```
plt.figure(figsize=(8, 5))
df["is_weekend"] = df["weekday"].apply(lambda x: 1 if x >= 5 else 0)
# Saturday/Sunday = 1

sns.barplot(x="is_weekend", y="national_demand", data=df,
palette="viridis")

plt.title("Weekend vs Weekday National Demand")
plt.xlabel("Is Weekend (0 = Weekday, 1 = Weekend)")
plt.ylabel("National Demand (MW)")
plt.grid()
plt.show()
```

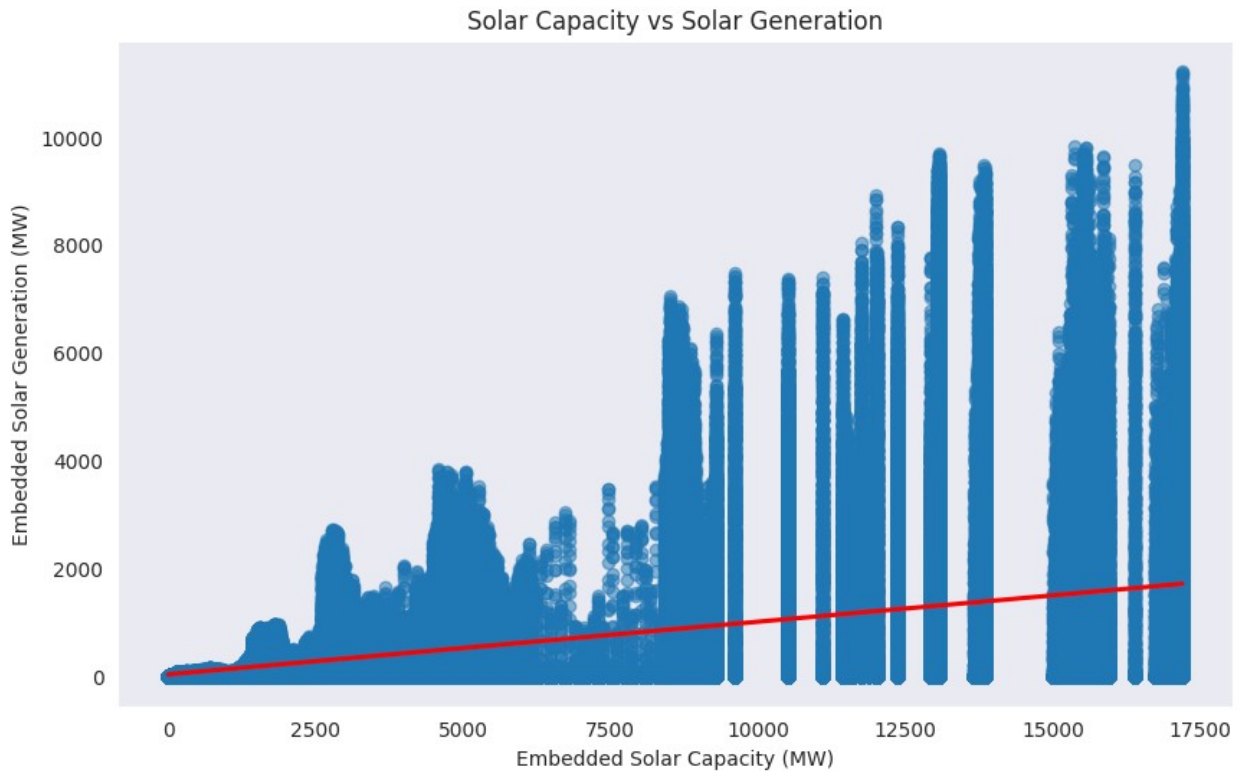


12. Impact of Embedded Solar Capacity on Solar Generation (Scatter Plot + Trendline)

See if **higher solar capacity** leads to **higher solar generation**.

```
plt.figure(figsize=(10, 6))
sns.regplot(x=df["embedded_solar_capacity"],
            y=df["embedded_solar_generation"], scatter_kws={"alpha":0.5},
            line_kws={"color": "red"})

plt.title("Solar Capacity vs Solar Generation")
plt.xlabel("Embedded Solar Capacity (MW)")
plt.ylabel("Embedded Solar Generation (MW)")
plt.grid()
plt.show()
```



□ Step 11: Final Report on Energy Demand & Generation Analysis

□ Summary of Key Findings & Insights

After analyzing the dataset, we extracted critical trends and patterns:

□ Energy Demand Trends:

- **National Demand** follows a clear **seasonal pattern**, with peaks in winter and drops in summer.
- Demand also fluctuates by **hour of the day**, with peaks during the morning and evening hours.

□ Renewable Energy Contribution:

- **Embedded Wind & Solar Generation** show strong variation based on weather conditions.
- **Solar generation** peaks during midday, while **wind generation** fluctuates more randomly.

□ Interconnector Energy Flow:

- Some **interconnectors (IFA, BritNed, Nemo, Moyle)** have both **positive (import)** and **negative (export)** values, showing cross-border energy trading.
- **Exports** are high when domestic generation exceeds demand, while **imports** increase during shortages.

□ Holidays vs. Weekdays Impact:

- **Lower demand on holidays** compared to normal weekdays.
 - Certain holiday seasons show a distinct drop in **total system demand**.
-
-

□ Key Predictive Insights

□ Future Demand:

- Forecast shows a **gradual increase** in national demand.
- Demand **spikes in winters** and **drops in summers**, aligning with historical trends.

□ Renewable Energy Trends:

- **Solar generation** expected to increase in summers.
- **Wind generation** remains **more volatile** due to unpredictable weather.

□ Interconnector Usage:

- **Higher imports** predicted in high-demand periods.
 - **Exports** likely during excess generation phases.
-

□ Final Takeaways

- The energy demand follows a **seasonal pattern** and is impacted by holidays.
- **Renewable energy** plays a major role but is dependent on weather conditions.
- **Forecasting models** help in predicting future energy needs for better planning.